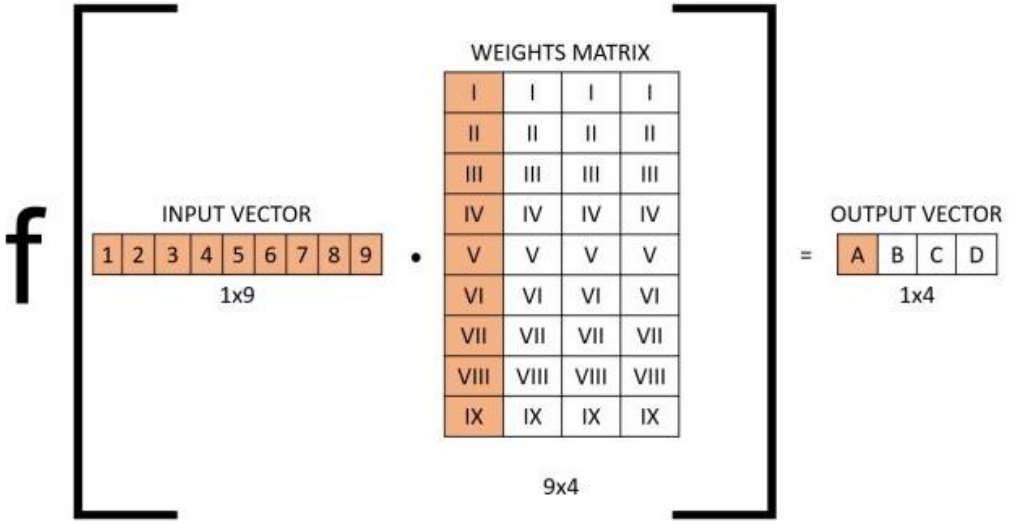
Deep Learning: Lab 2

Convolutional Neural Networks

Recap on fully connected networks

- In Lab 1: trained **fully connected network (FCN)**
- FCN processes data as a vector $\mathbb{R}^n$ (n = input dimension of data)
- Every layer of a FCN is a matrix that maps $\mathbb{R}^i \rightarrow \mathbb{R}^o$ (i = input dimension of layer, o = output dimension)
- This means $i * o$ parameters



Practicalities

- Hyperparameter tuning is still relevant but time consuming, so we don’t consider it in this lab
- You will have to use GPU to run the models, otherwise training takes very long
- Will show how to use GPU in other slide deck

Image processing use case



- Cifar-10 dataset: 10 classes
- Low resolution RGB image of 32 x 32 pixels
- Every pixel stores RGB values (3 values from 0…255)
- This means 32*32*3 values, to pass to FCN: flattened into a vector $\mathbb{R}^{3072}$
- FCN has no grasp on proximity of pixels: all color values and positions are processed independently
- **<u>Very inefficient!</u>**

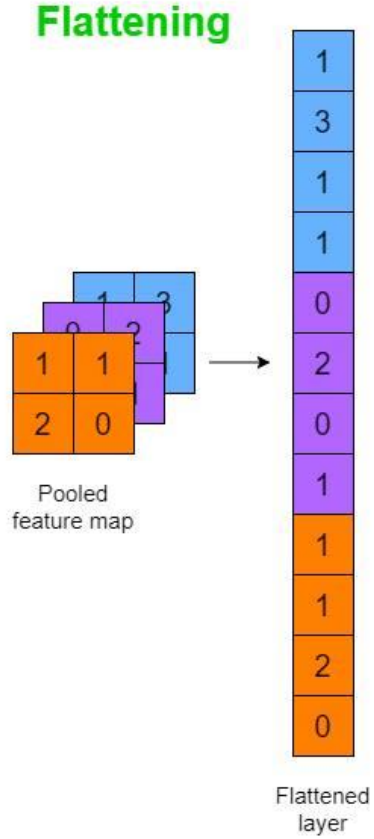
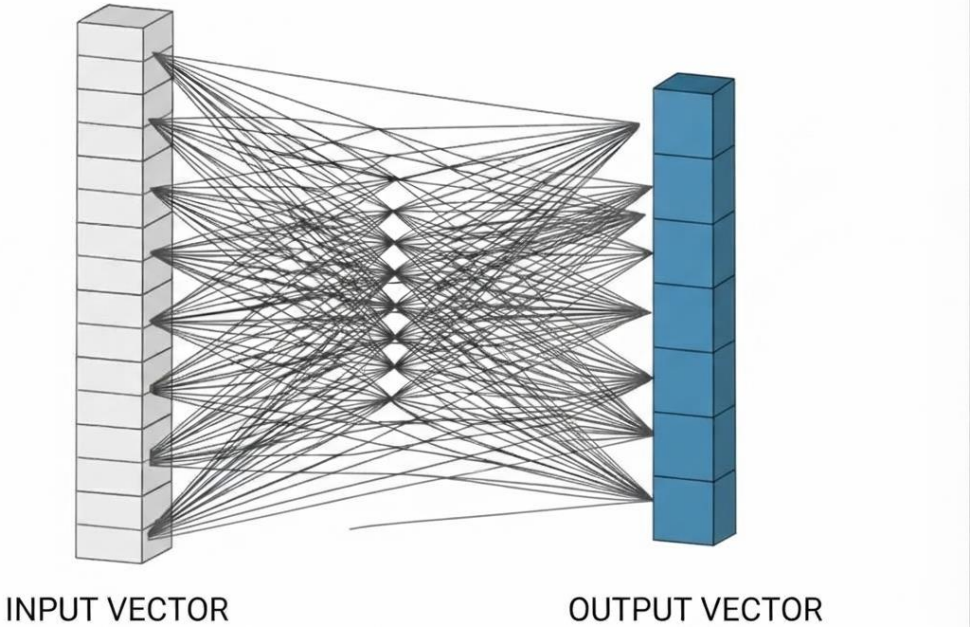


Image processing use case

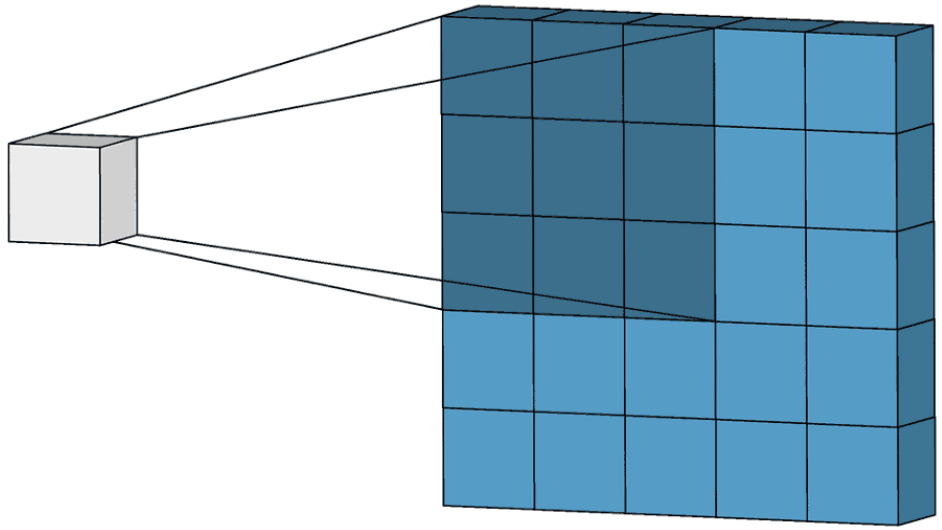
Fully connected

- Every input has its own parameters
- Output is structureless



Convolutional

- Reuse of parameters over all locations
- Output preserves local structure of input



Building blocks of Convolutional Networks

Conv1d

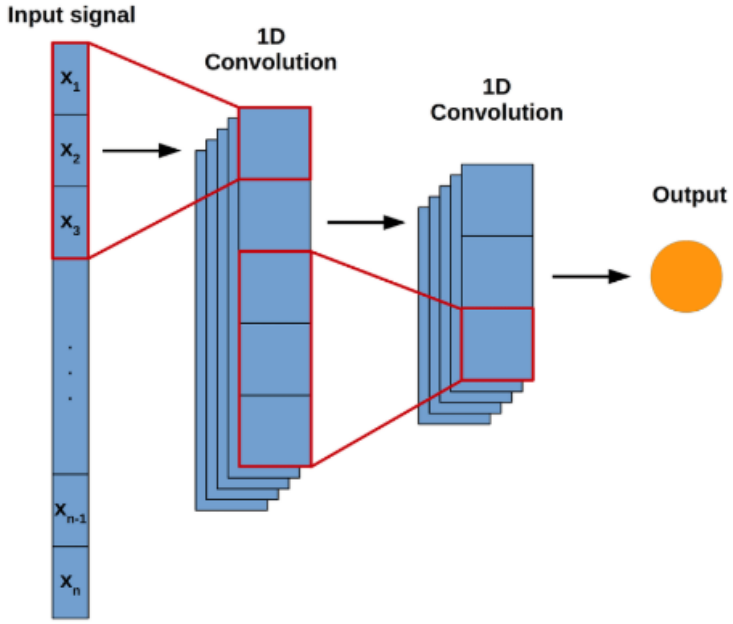
class torch.nn.Conv1d(*in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True, padding_mode='zeros', device=None, dtype=None*) [source]

Applies a 1D convolution over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C_{in}, L) and output $(N, C_{\text{out}}, L_{\text{out}})$ can be precisely described as:

$$\text{out}(N_i, C_{\text{out}_j}) = \text{bias}(C_{\text{out}_j}) + \sum_{k=0}^{C_{\text{in}}-1} \text{weight}(C_{\text{out}_j}, k) \star \text{input}(N_i, k)$$

where $\star$ is the valid cross-correlation operator, N is a batch size, C denotes a number of channels, L is a length of signal sequence.



Conv2d

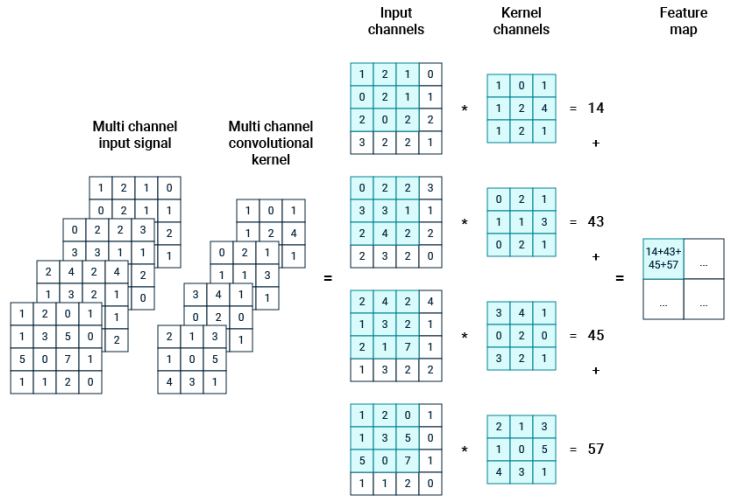
class torch.nn.Conv2d(*in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True, padding_mode='zeros', device=None, dtype=None*) [source]

Applies a 2D convolution over an input signal composed of several input planes.

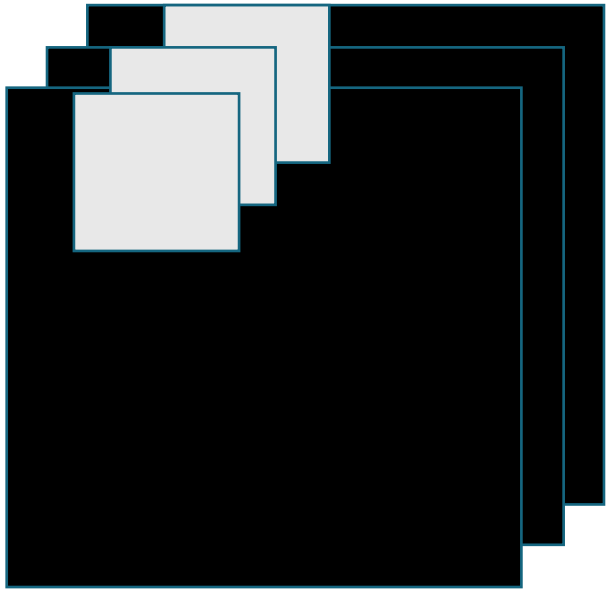
In the simplest case, the output value of the layer with input size (N, C_{in}, H, W) and output $(N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ can be precisely described as:

$$\text{out}(N_i, C_{\text{out}_j}) = \text{bias}(C_{\text{out}_j}) + \sum_{k=0}^{C_{\text{in}}-1} \text{weight}(C_{\text{out}_j}, k) \star \text{input}(N_i, k)$$

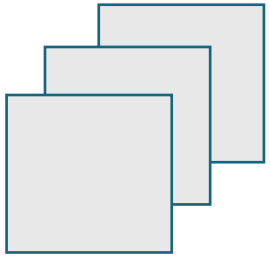
where $\star$ is the valid 2D cross-correlation operator, N is a batch size, C_{in} and C_{out} correspond to `in_channels` and `out_channels` respectively, H and W are the input height and width in pixels. See the Shape section below for how H_{out} and W_{out} are derived from `kernel_size`, `stride`, `padding`, and `dilation`.



Convolution with 1 filter



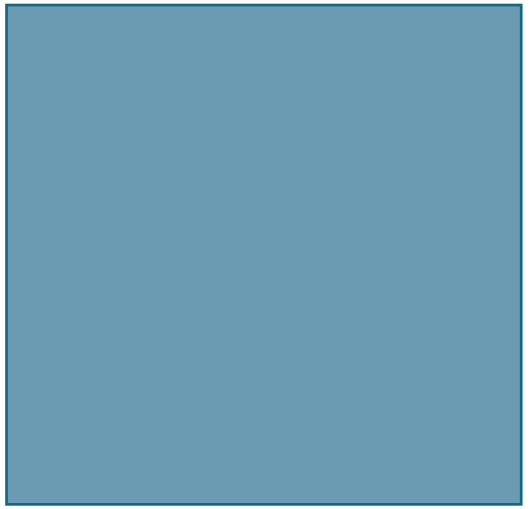
32 x 32 x **3**



3 x 3 x **3** filter

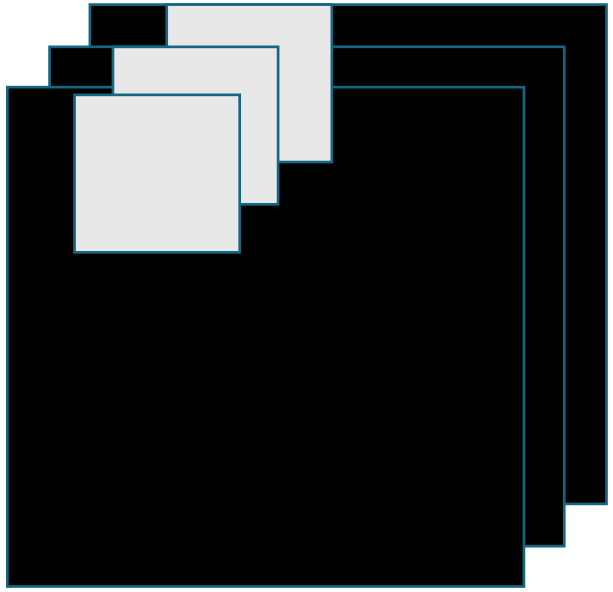
Convolving over the entire input volume

Activation map

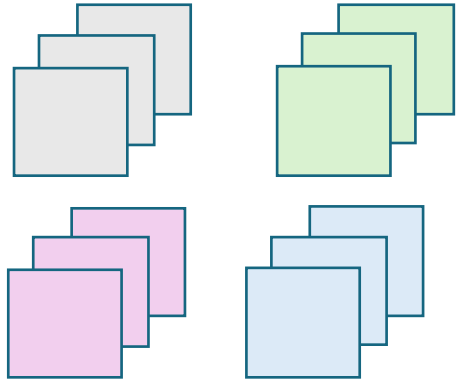


28 x 28 x 1

Convolution with multiple filters



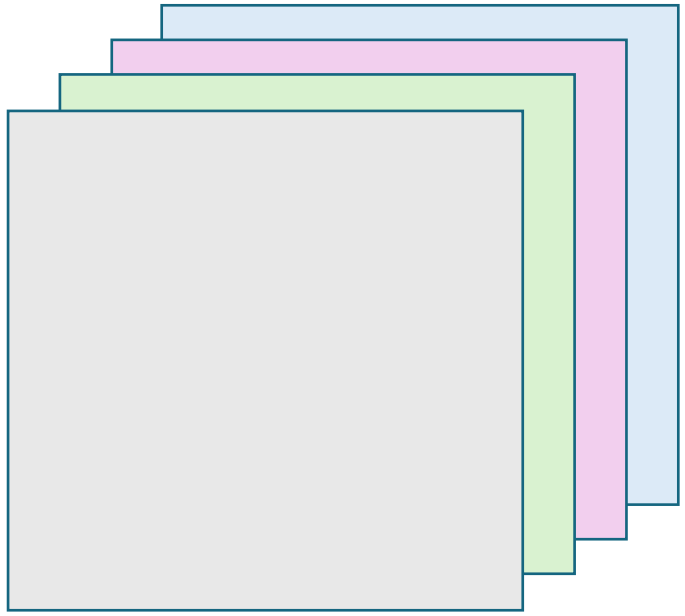
32 x 32 x 3



Convolve with multiple independent filters.

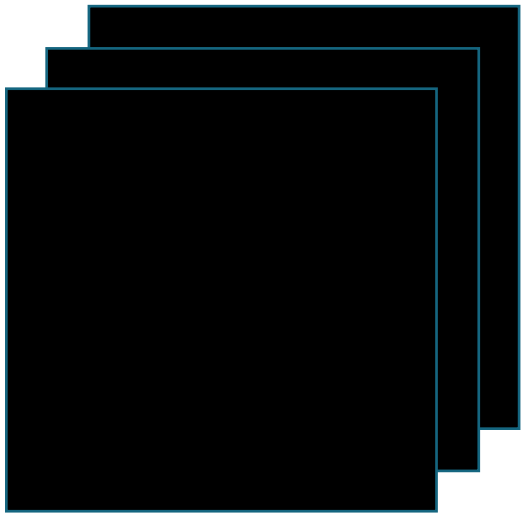
For instance, 4 filters

Activation map



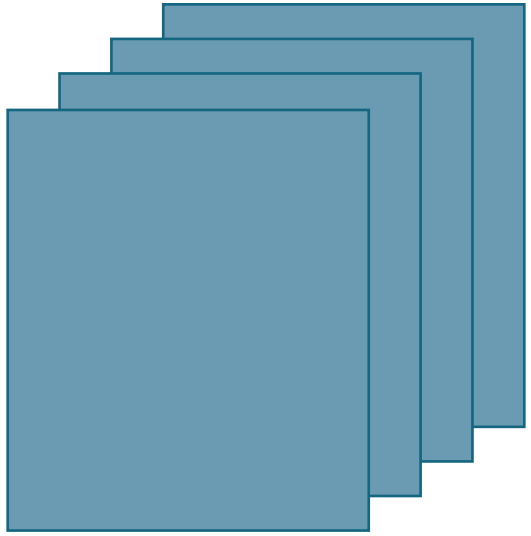
28 x 28 x 4

Consecutive convolutions



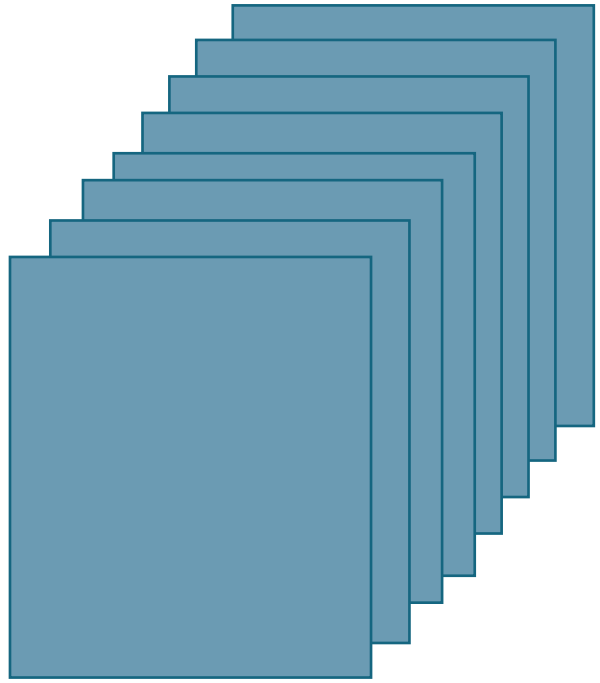
32 x 32 x 3

4 filters
5x5x3



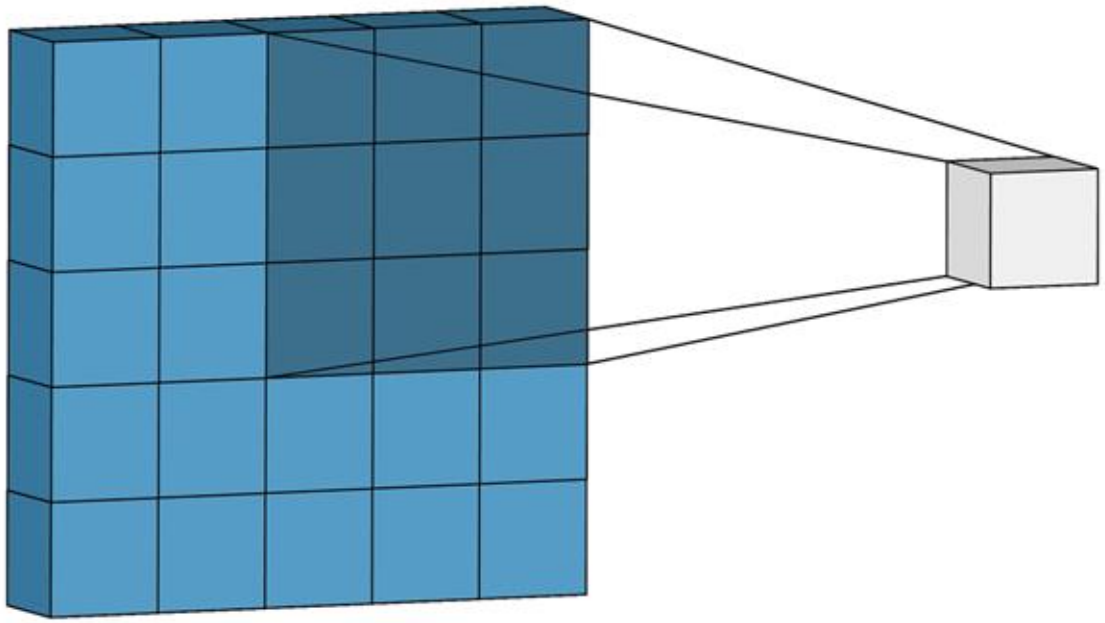
28 x 28 x 4

8 filters
5x5x4



24 x 24 x 8

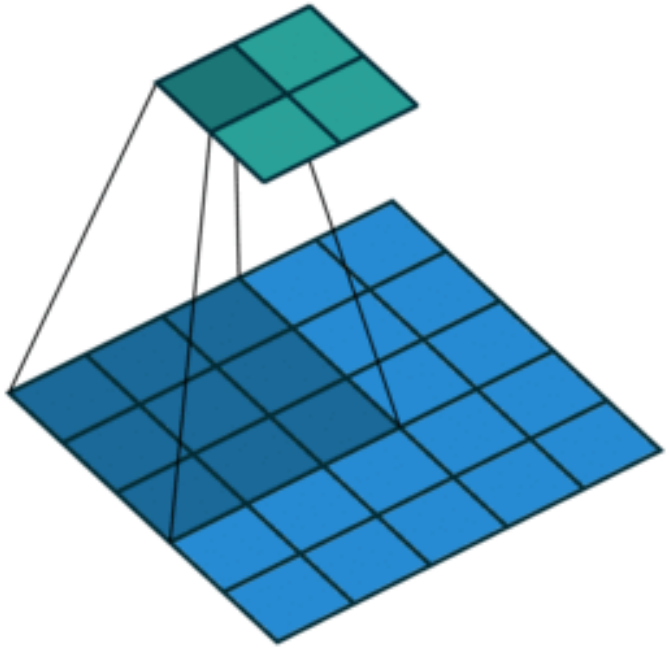
Filter: 3 x 3



Input: 5 x 5

3 x 3

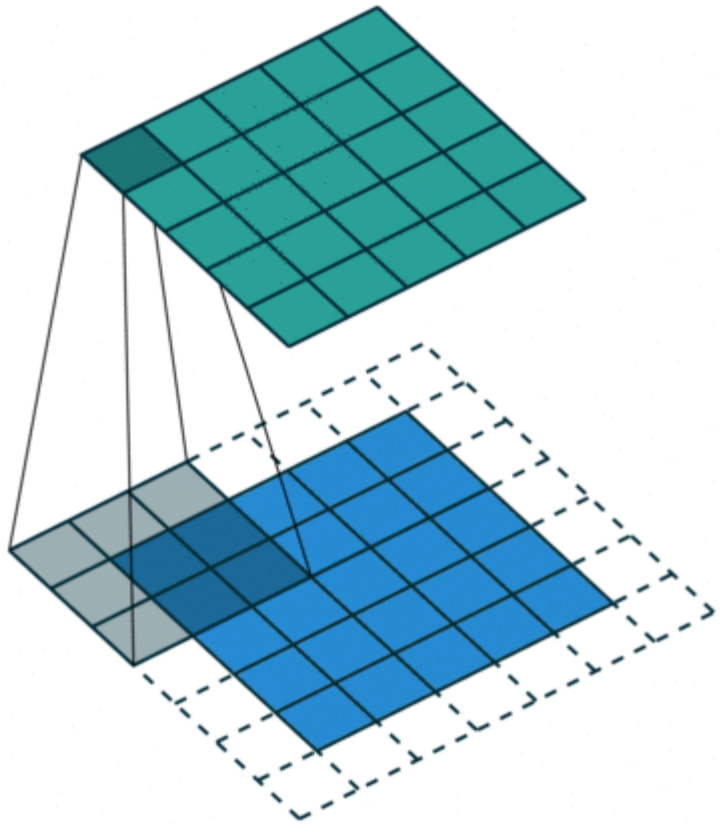
Output: 2 x 2



Filter: 3 x 3

Stride: 2

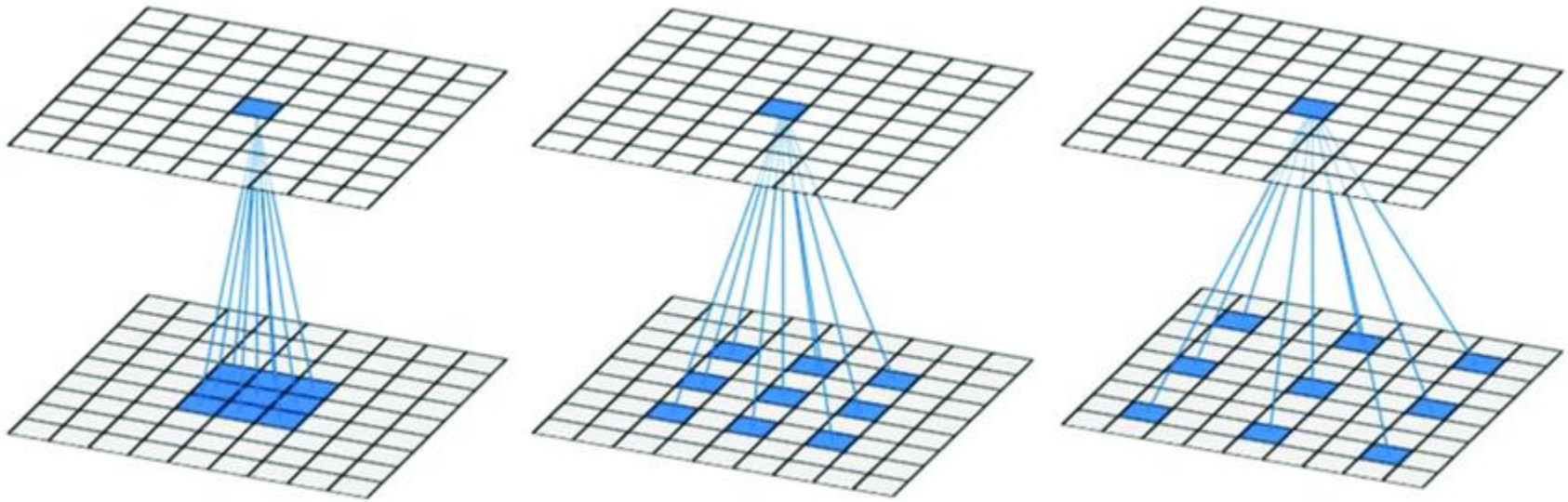
Input: 5 x 5



Output: 5 x 5

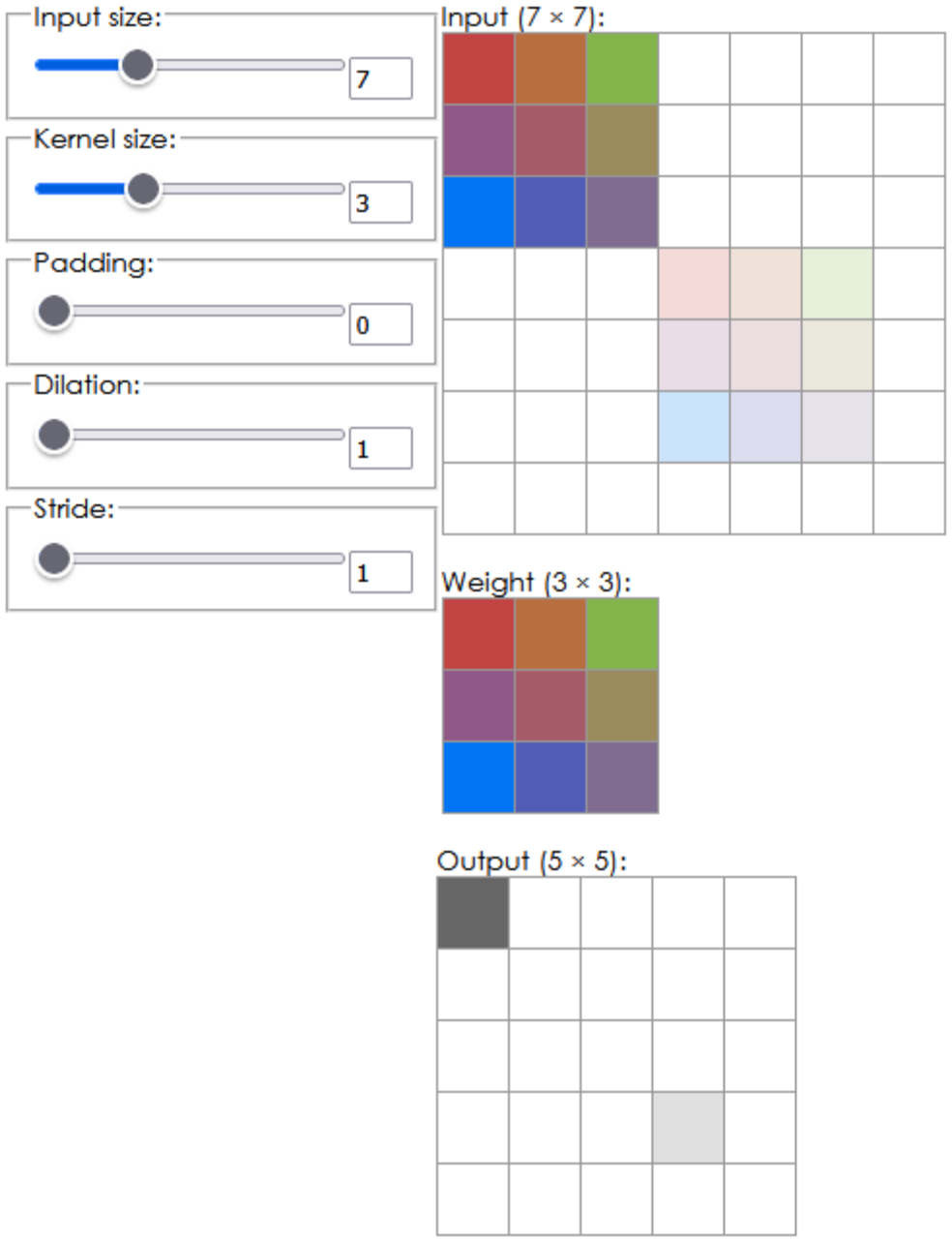
Filter: 3 x 3
Stride: 1

Input: 5 x 5 (7 x 7)


dilation=1
dilation=2
dilation=3

Convolution parameters

- Kernel
- Padding
- Dilation
- Stride

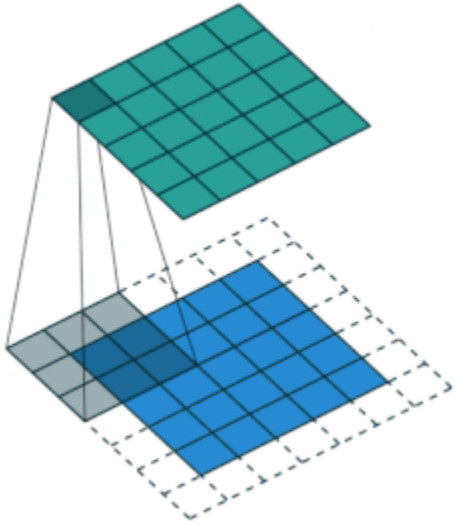


https://ezyang.github.io/convolution-visualizer/index.html

Padding in pytorch

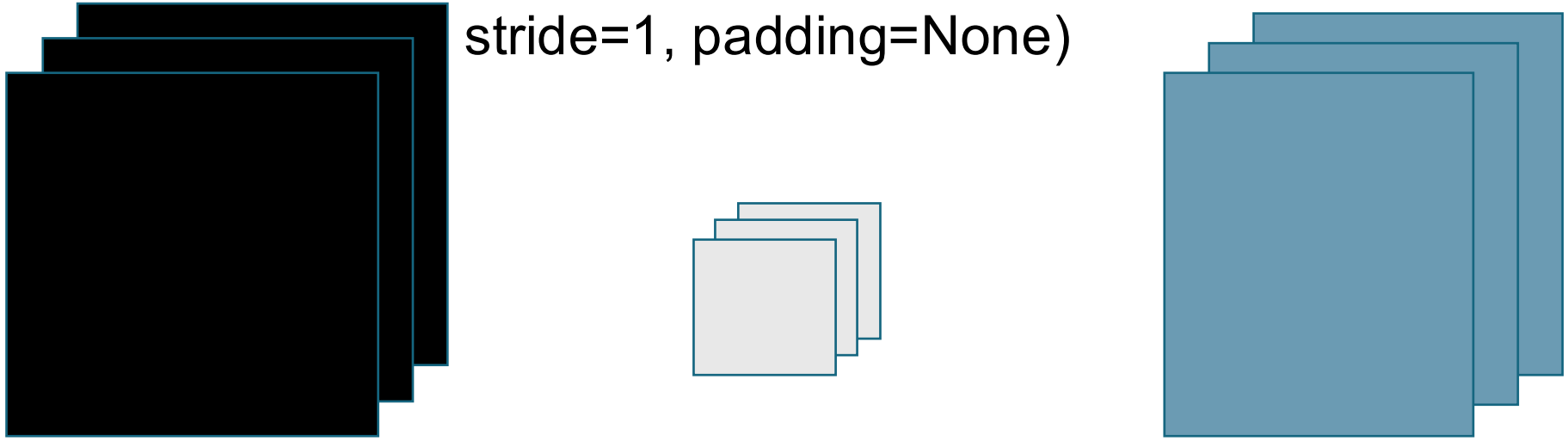
- `padding` controls the amount of padding applied to the input. It can be either a string {'valid', 'same'} or an int / a tuple of ints giving the amount of implicit padding applied on both sides.

- Often you maintain same resolution as input, so you specify 'same' padding, automatically adds the right amount of 0's around input



Example: how many parameters?

torch.nn.Conv2d(*in_channels*=**3**,
out_channels=10, kernel_size= 3,
stride=1, padding=None)



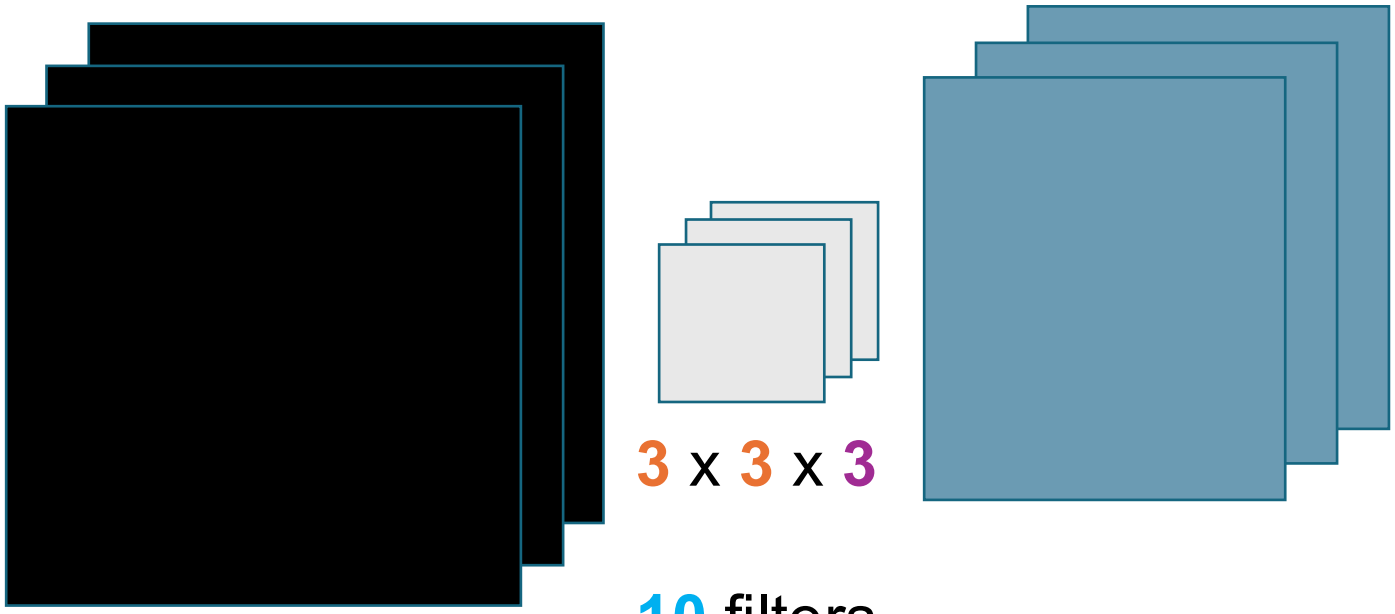
32 x 32 x **3**

? x ? x **?**

? x ? x **?**

Implemented

torch.nn.Conv2d(*in_channels*=**3**, *out_channels*=**10**, *kernel_size*= **3**, stride=**1**, padding=**None**)



Input shape:

- 32 x 32 x 3

Number of parameters:

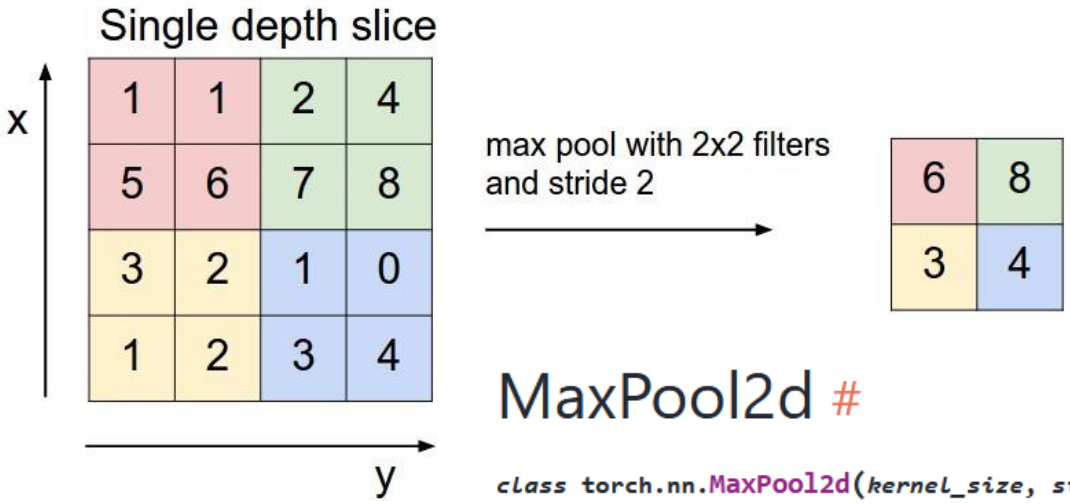
- 10 filters of size 3 * 3 * 3 (=27)
- Filter parameters = 10 x 27
- And bias term per filter = 10
- Total params = 280

Output shape:

- *kernel* 3; no padding; *stride* 1; 10 *out_channels*
- 30 by 30 possible positions
- Output shape = 30 x 30 x 10

Pooling

Sometimes you want to downscale resolution: pooling



MaxPool2d

class **torch.nn.MaxPool2d**(***kernel_size***, ***stride***=*None*, ***padding***=*0*, ***dilation***=*1*, ***return_indices***=*False*, ***ceil_mode***=*False*) [source]

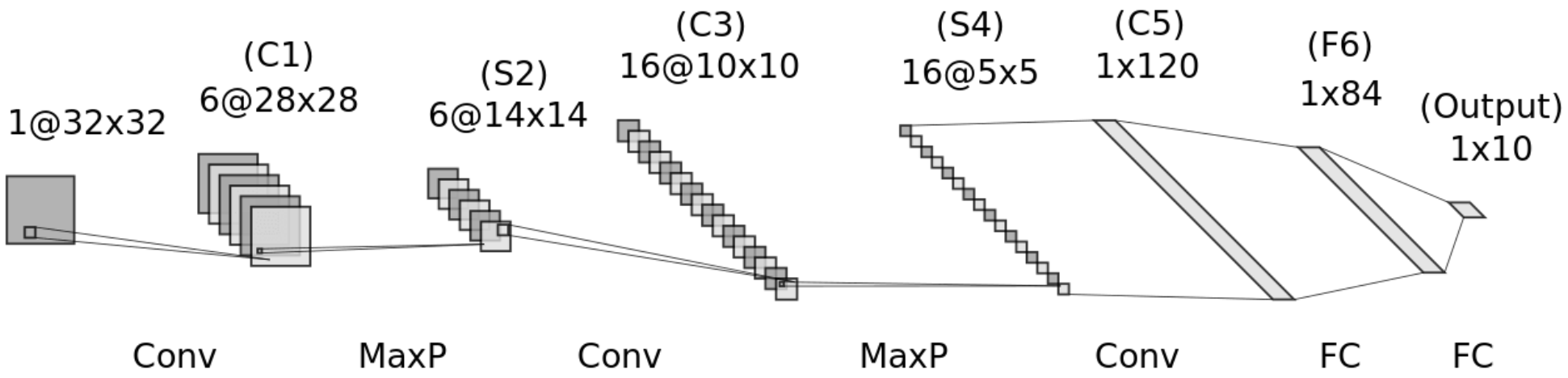
Applies a 2D max pooling over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C, H, W), output (N, C, H_{out}, W_{out}) and `kernel_size` (kH, kW) can be precisely described as:

$$out(N_i, C_j, h, w) = \max_{m=0,\ldots,kH-1} \max_{n=0,\ldots,kW-1} \text{input}(N_i, C_j, \text{stride}[0] \times h + m, \text{stride}[1] \times w + n)$$

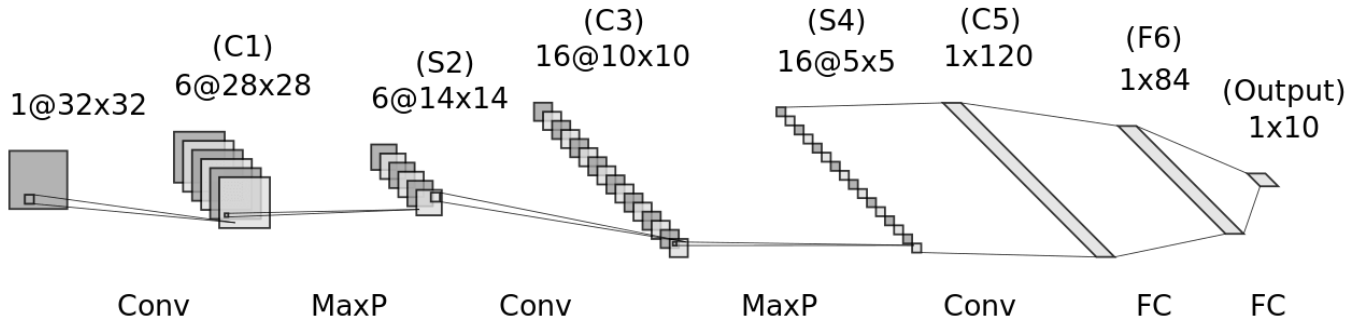
If `padding` is non-zero, then the input is implicitly padded with negative infinity on both sides for `padding` number of points. `dilation` controls the spacing between the kernel points. It is harder to describe, but this link has a nice visualization of what `dilation` does.

Basic Convolution architecture: LeNet-5



Convolution layers extract patterns, FC layers reason about the patterns that are present

Step 1 in this lab:



- Implement this Convolution architecture on RGB images
- Understand the parameters of the Conv layer:

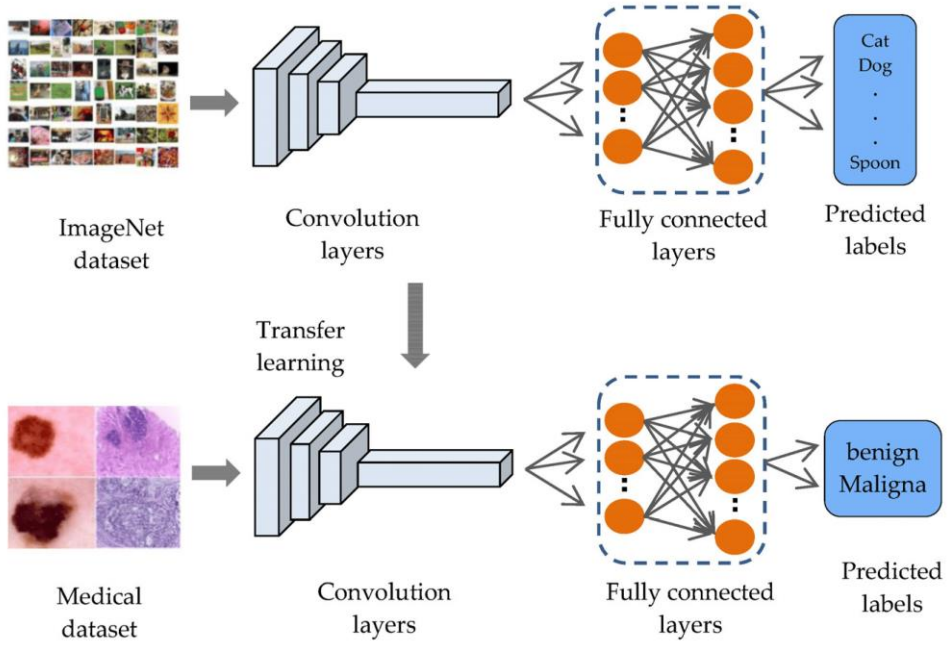
Conv2d

```
class torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0,
dilation=1, groups=1, bias=True, padding_mode='zeros', device=None, dtype=None) #  [source]
```

- Visualization of different conv operations: https://github.com/vdumoulin/conv_arithmetic

Transferring knowledge between models

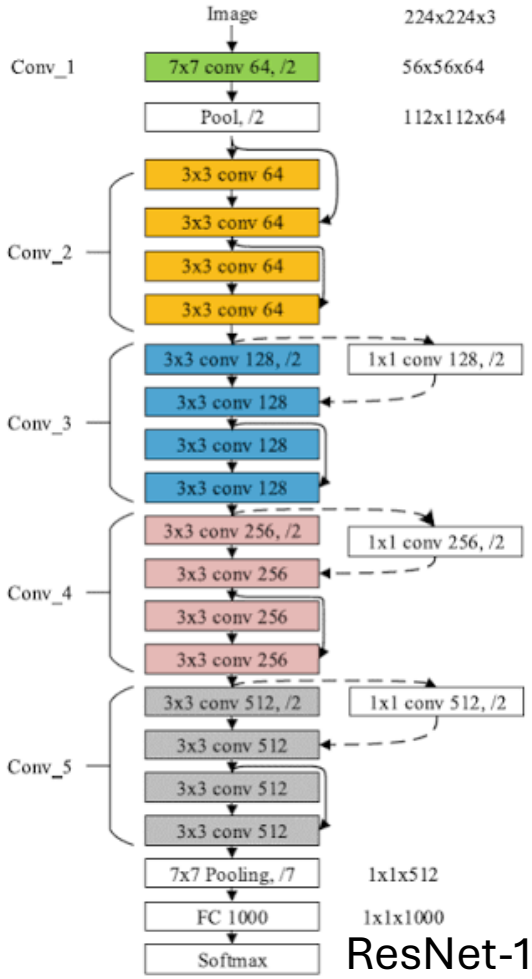
- Training image recognition networks from scratch is not common anymore
- It is better to use another network pretrained on very large datasets for generic pattern recognition in the beginning of the network
- This is known as **transfer learning**
  1. Take over pattern recognition (Conv) params
  2. Train reasoning part (FCN) on specific dataset



ResNet models



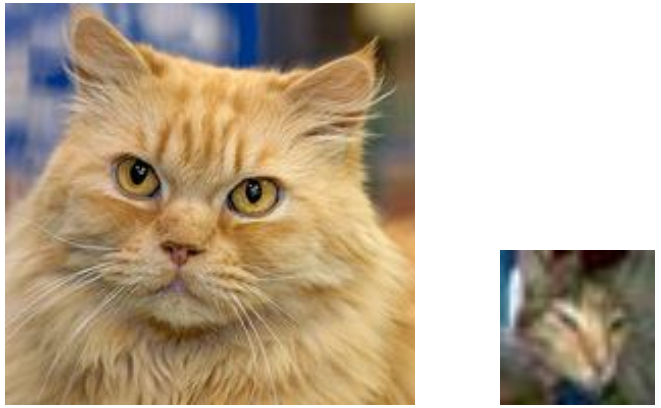
- Model trained on ImageNet dataset: ~1.5 million images with 1000 classes
- Trained to classify the item in the image
- Different ResNet sizes (total layers XX in the name: ResNet-XX)
- ImageNet is very versatile, so the models can process images from other datasets
- ResNet was a breakthrough on very deep modeling due to skip connections



ResNet-18

How to use pretrained backbones?

- Image scale mismatch:
  - ImageNet resized for ResNet: 224 x 224 x 3
  - Cifar-10 dataset: 32 x 32 x 3
- Convolutions work on any size so technically you can pass the data
- Mismatch in training scale and test scale introduce a **domain shift**
- Upscaling is a good idea, not necessary to 244, 64 can also work (less computation needed)



ImageNet vs Cifar-10 image

Upsample

class **torch.nn.Upsample**(***size***=*None*, ***scale_factor***=*None*, ***mode***='nearest', ***align_corners***=*None*, ***recompute_scale_factor***=*None*) [source]

Upsamples a given multi-channel 1D (temporal), 2D (spatial) or 3D (volumetric) data.

The input data is assumed to be of the form *minibatch x channels x [optional depth] x [optional height] x width*. Hence, for spatial inputs, we expect a 4D Tensor and for volumetric inputs, we expect a 5D Tensor.

The algorithms available for upsampling are nearest neighbor and linear, bilinear, bicubic and trilinear for 3D, 4D and 5D input Tensor, respectively.

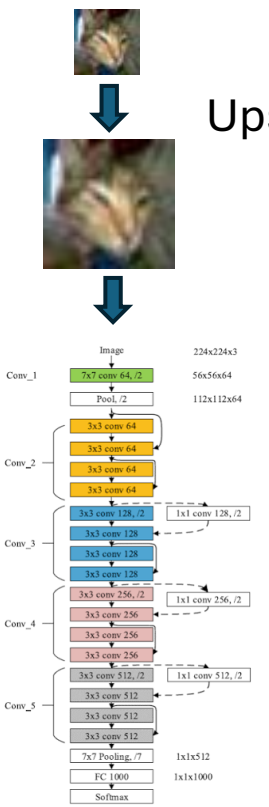
One can either give a `scale_factor` or the target output `size` to calculate the output size. (You cannot give both, as it is ambiguous)

Example use:

```
import torch.nn as nn
from torchvision.models import resnet18

model = nn.Sequential(
    nn.Upsample(scale_factor=2, mode='bilinear', align_corners=False),
    resnet18(weights="IMAGENET1K_V1"),
)

x = torch.randn(8, 3, 32, 32)
print(model(x).shape)  # torch.Size([8, 1000])
```



Reusing a subset of a model

- Using all model weights is not always necessary
- ResNet-18 has 11,7 million parameters: 75% are in 4th block of convolutions
- Reduces compute for simpler problems

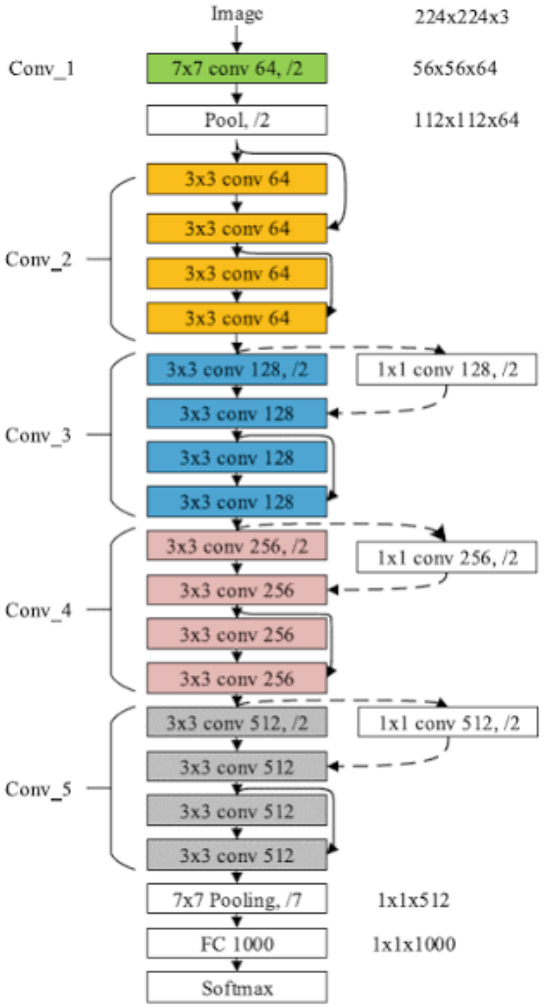
```
def _forward_impl(self, x: Tensor) -> Tensor:
    # See note [TorchScript super()]
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.maxpool(x)

    x = self.layer1(x)
    x = self.layer2(x)
    x = self.layer3(x)
    x = self.layer4(x)

    x = self.avgpool(x)
    x = torch.flatten(x, 1)
    x = self.fc(x)

    return x
```

=



Changing the model (not going through layer 4)

- Create a Model subclass and write custom forward pass:

```python
class MiniResNet18(nn.Module):
    def __init__(self):
        super().__init__()
        self.r = resnet18(weights="IMAGENET1K_V1")
        self.r.fc = nn.Linear(256, 1000) # new head, layer 3 outputs 256 channels in ResNet-18

    def forward(self, x):
        x = self.r.conv1(x)
        x = self.r.bn1(x)
        x = self.r.relu(x)
        x = self.r.maxpool(x)

        x = self.r.layer1(x)
        x = self.r.layer2(x)
        x = self.r.layer3(x)

        x = torch.flatten(x, 1)
        x = self.r.fc(x)

        return x
```

Adding a custom classification layer

- Resnet has a single fc-layer from conv output to multi class prediction
- This can be changed to anything we like, ex:

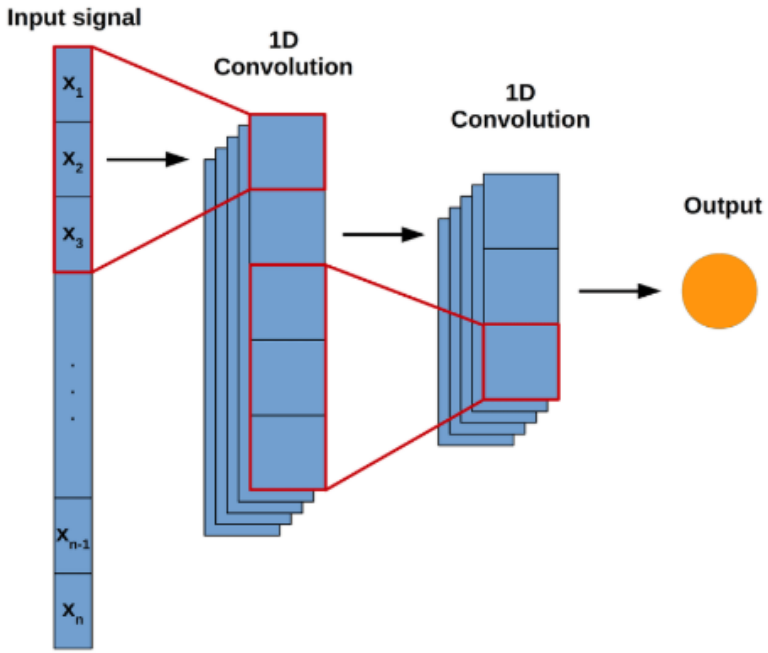
```
class MiniResNet18(nn.Module):
    def __init__(self,):
        super().__init__()
        self.r = resnet18(weights="IMAGENET1K_V1")
        self.r.fc = nn.Sequential(
            nn.Linear(256, 32),
            nn.ReLU(),
            nn.Linear(32, 10),
        )
```

Step 2 of Lab:

- Implement ResNet Subclass
- Train only fc part, conv part frozen
- Compare performance with your conv net trained from scratch

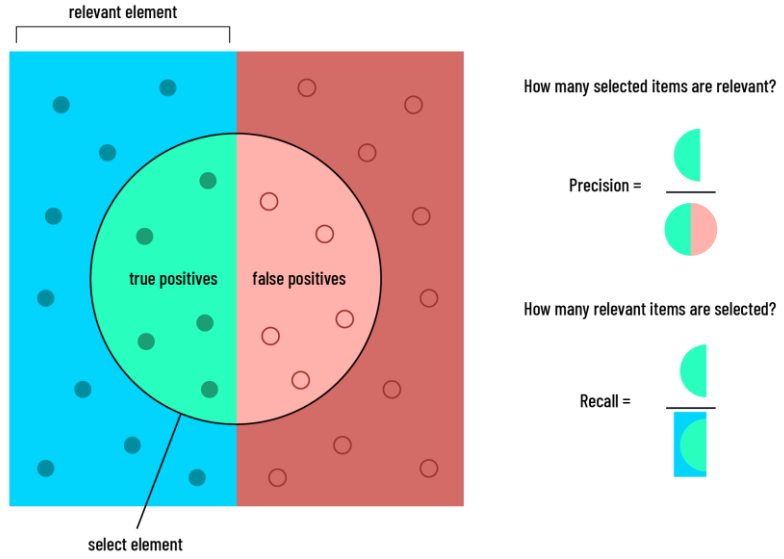
Models for signals

- We saw models for images, they typically use Conv2D because pixels in a region are correlated and local information is meaningful
- In signals, channels are typically treated independently: Conv1D



Models for medical applications

- When using data from subjects, **split train and test by subjects**
- If a subject has data in train and test sets, the performance estimates could be too optimistic
- Medical data is often imbalanced (ex. rare disease), it skews accuracy metrics
- Use precision, recall and F1 when evaluating



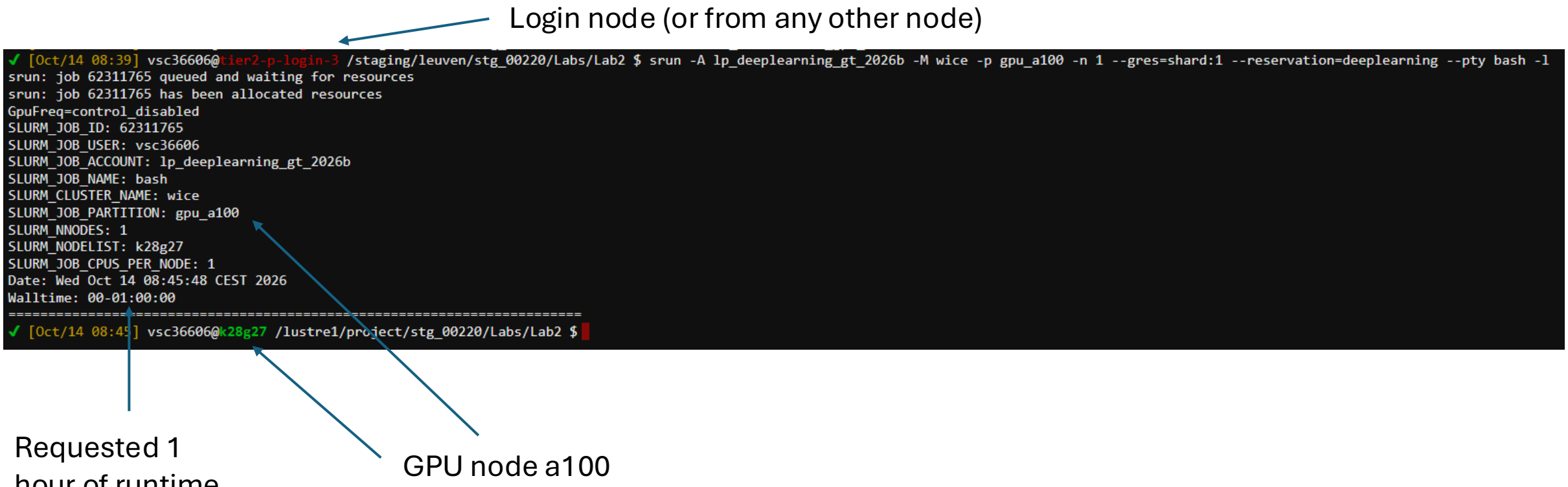
Evaluating models in medical applications

- Leave one subject out approach
- Train model on all combinations of N-1 subjects and take the average of the performances
- Time consuming so not done in this lab, but be aware of this method



Running code on GPU

- File with GPU terminal launch code for this lab in /staging/leuven/stg_00220/Labs/Lab2/start_interactive_gpu_terminal.sh



nvidia-smi to inspect gpu

```
✓ [Oct/14 08:50] vsc36606@tier2-p-login-3 /vsc-hard-mounts/leuven-user/366/vsc36606 $ nvidia-smi
Wed Oct 14 08:50:40 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.178.04             Driver Version: 580.178.04     CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Quadro P6000                   Off |   00000000:AF:00.0 Off |                  Off |
| 19%   36C    P0             55W /  250W |       0MiB /  24576MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

```
✓ [Oct/14 08:45] vsc36606@k28g27 /lustre1/project/stg_00220/Labs/Lab2 $ nvidia-smi
Wed Oct 14 08:49:46 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 615.71.09              KMD Version: 615.71.09     CUDA UMD Version: 13.4     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA A100-SXM4-80GB          On  |   00000000:18:00.0 Off |                    0 |
| N/A   34C    P0             64W /  500W |       0MiB /  81920MiB |      0%      Default |
|                                         |                        |             Disabled |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

Also possible with slurm file

/staging/leuven/stg_00220/Labs/Lab2/ lab_reservation_GPU.slurm

```
 GNU nano 5.6.1
#!/bin/bash -l
#SBATCH --account=lp_deeplearning_gt_2026b
#SBATCH --clusters=wice
#SBATCH --partition=gpu_a100
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --gres=shard:1
#SBATCH --reservation=deeplearning

# load relevant modules first
module load PyTorch-bundle/2.9.1-foss-2025a-CUDA-12.8.0-whl

# run your file: python "path\to\file"
```

Currently only load pytorch module and runs nothing

```
✓ [Oct/14 08:52] vsc36606@tier2-p-login-3 /staging/leuven/stg_00220/Labs/Lab2 $ sbatch lab_reservation_GPU.slurm
Submitted batch job 62311776 on cluster wice
✓ [Oct/14 08:52] vsc36606@tier2-p-login-3 /staging/leuven/stg_00220/Labs/Lab2 $
```

Try it out!